

Universidad Nacional Autónoma de México

MODELADO Y PROGRAMACIÓN

Reporte

*José Galaviz
Brenda Ayala Flores
José Eduardo Cruz Campos*

Alumno:

Hazel Torres Nava
319158496
hazelt@ciencias.unam.mx



Índice

1. Definición del problema	2
2. Arsenal	2
3. Análisis del problema	2
4. Selección de la mejor alternativa	3
5. Pseudocódigo de la solución principal	3

1. Definición del problema

Para este proyecto el problema principal consiste en desarrollar un programa capaz de recibir una imagen, identificar las figuras que se encuentran dentro de ella y clasificarlas según su forma y color.

- **Entrada:** El sistema recibe la ruta de un archivo de imagen, preferentemente en formato `.bmp`, mediante los argumentos de la línea de comandos.
- **Salida:** El programa genera un reporte en la consola en forma de tabla, donde se muestra cada figura encontrada, su categoría correspondiente (C, T, O, X) y su color representado en código hexadecimal. En caso de que no se encuentren figuras o exista algún problema con el archivo, el programa muestra un mensaje indicando lo que ocurrió.

2. Arsenal

Para realizar el proyecto decidí utilizar Python, principalmente porque es un lenguaje con el que ya estoy acostumbrado a trabajar y porque facilita bastante el manejo de estructuras como listas y conjuntos.

- **Lenguaje:** Se utilizó Python debido a su sintaxis y al manejo que tiene de estructuras de datos como conjuntos (`set`) y listas. También resulta práctico para hacer una herramienta que se ejecute directamente desde la línea de comandos.
- **Bibliotecas:**
 - **Pillow (PIL):** Se utilizó para abrir la imagen, convertirla a formato RGB y obtener la matriz de píxeles. También se utilizó para poder generar el banco de pruebas. Decidí utilizar esta biblioteca para no tener que depender de herramientas más grandes como OpenCV solamente para leer y manipular imágenes BMP.
 - **math:** Se utilizó principalmente para realizar las operaciones matemáticas necesarias para los algoritmos, como el cálculo de hipotenusas, arcotangentes y desviaciones estándar.
 - **sys:** Se utilizó para obtener los argumentos proporcionados desde la terminal y para controlar la salida del programa mediante `sys.exit()`.

3. Análisis del problema

Uno de los supuestos más importantes que tomé para realizar el programa es que el píxel ubicado en la posición inicial `[0,0]` corresponde al fondo de la imagen. Esto es importante porque a partir de ese color se pueden distinguir las figuras del fondo.

También encontré algunos casos límite que era necesario tomar en cuenta:

- Archivos con extensiones incorrectas. En este caso se muestra una advertencia, pero el programa intenta procesar el archivo.

- Rutas inexistentes o archivos dañados. Estos casos se manejan mediante bloques `try-except`, de forma que el programa pueda mostrar un mensaje entendible en lugar de terminar mostrando un error difícil de interpretar.
- Imágenes de un solo color. Si toda la imagen tiene el mismo color que el píxel $[0,0]$, entonces no existen figuras que identificar. En este caso el programa informa al usuario y termina la ejecución.

En cuanto a los requisitos funcionales, el programa tiene que obtener los colores RGB de los píxeles y convertirlos a hexadecimal, agrupar los píxeles contiguos que pertenezcan a una misma figura, analizar el contorno de cada agrupación y finalmente clasificar la figura como Cuadrilátero, Triángulo, Círculo u Otro.

Como requisito no funcional considero principalmente la robustez del programa. No quiero que ante cualquier problema con el archivo se muestre un *stack trace* que no sea útil para el usuario, sino un mensaje que indique qué ocurrió. También se debe cuidar el uso de memoria, principalmente para evitar un *StackOverflow* cuando se procesen figuras demasiado grandes.

4. Selección de la mejor alternativa

Para la detección de las figuras decidí utilizar una búsqueda en profundidad (DFS) iterativa utilizando una estructura de datos de pila. Preferí esta opción sobre una implementación recursiva porque una figura puede estar formada por una cantidad bastante grande de píxeles y una recursión demasiado profunda podría provocar problemas de memoria. De esta forma se mantiene el funcionamiento del DFS, pero sin depender de la pila de llamadas del programa.

Para la clasificación geométrica decidí utilizar un algoritmo de **Análisis de Perfil Radial**. Preferí esta alternativa sobre utilizar simplemente una caja delimitadora (*bounding box*), ya que el perfil radial permite que la clasificación sea más independiente de la rotación de la figura.

El algoritmo primero encuentra el centroide de la figura y posteriormente obtiene su frontera exterior. A partir del centro se utilizan 72 “rebanadas” de 5 grados para medir la distancia desde el centro hasta los bordes de la figura.

Con estas distancias puedo obtener información sobre la forma. Por ejemplo, para determinar si una figura es un círculo se calcula la desviación estándar de las distancias. Si esta se encuentra dentro de una tolerancia del 12%, se considera que la figura es un círculo. Para los polígonos se buscan los picos que sobresalen del radio promedio en más de un 5%. La cantidad de estos picos permite determinar la cantidad de vértices. Si se encuentran 3 picos se clasifica como triángulo (T) y si se encuentran 4 como cuadrilátero (C). En cualquier otro caso se clasifica como otro (X).

5. Pseudocódigo de la solución principal

El pseudocódigo que utilicé para representar el funcionamiento general del programa es el siguiente:

INICIO

Obtener `ruta_imagen` desde los argumentos de la consola

Validar extensión del archivo

INTENTAR:

Cargar imagen y extraer matriz de píxeles
color_fondo = píxel[0, 0]

// Fase de Segmentación (DFS Iterativo)

PARA CADA x, y EN matriz_píxeles:

SI píxel[x, y] != color_fondo Y NO ESTÁ EN visitados:

Agregar a pila y aplicar búsqueda DFS explorando vecinos

Guardar conjunto de píxeles y color hexadecimal como 'Figura'

// Fase de Clasificación (Perfil Radial)

PARA CADA Figura EN figuras_encontradas:

Extraer contorno (píxeles que tocan el fondo)

Calcular centroide (cx, cy)

Calcular distancias de todos los puntos del contorno al centroide

SI desviación_estándar de distancias < 12%:

Clasificación = 'O' (Círculo)

SINO:

Dividir la figura en 72 ángulos (rebanadas)

Suavizar las distancias usando media móvil

Contar número de picos mayores al 105% del radio promedio

SI picos == 3: Clasificación = 'T'

SI picos == 4: Clasificación = 'C'

DE LO CONTRARIO: Clasificación = 'X'

Imprimir Categoría y Color

ATRAPAR ERRORES:

Imprimir mensaje de error claro al usuario

Salir del programa

FIN